Report And Analysis Of Rabin Karp Algorithm

Vatsa Joshi

INDEX:

Introduction:

The Rabin-Karp Algorithm is a string-searching algorithm that efficiently finds patterns within a text. This algorithm is particularly useful when you need to search for multiple patterns in a text or when you have a single pattern that you want to find multiple times within a larger body of text. It achieves this efficiency by utilizing a technique called hashing.

Hashing transforms a string into a numerical value, known as a hash value or hash code, which uniquely represents the string. By comparing these hash values instead of the strings themselves, the algorithm can quickly determine whether two strings are identical. This makes the Rabin-Karp Algorithm especially effective for searching multiple patterns at once.

Why is it important?

The Rabin-Karp Algorithm is crucial in various applications where quick and efficient pattern matching is needed. For example:

Text Processing: It helps in quickly finding words or phrases in large texts, such as documents, articles, or books.

Plagiarism Detection: The algorithm is used to identify copied content by comparing large sets of documents for overlapping text.

Bioinformatics: In DNA sequence analysis, it helps in matching patterns within genetic codes.

Search Engines: Search engines use it to match search queries with a large database of web pages.

Digital Forensics: It aids in finding specific patterns within large datasets during forensic investigations.

Where is it used?

The Rabin-Karp Algorithm finds applications in various fields, including:

Text Editing Software: For implementing 'Find' features that locate specific words or phrases.

Database Systems: For querying large datasets for specific patterns.

Network Security: For detecting patterns of malicious activity in network traffic.

Data Compression: For finding repeated patterns that can be compressed.

History of the Rabin-Karp Algorithm:

The Rabin-Karp Algorithm was developed by Richard M. Karp and Michael O. Rabin in 1987. Their groundbreaking work introduced the concept of using hashing for string searching, which significantly improved the efficiency of pattern matching algorithms. Their method has since become a foundational tool in computer science, influencing various applications and other algorithms.

Applications:

The Rabin-Karp algorithm benefits various fields due to its efficient pattern matching capabilities. Here are its applications:

Text Searching: Utilized in text editors and word processors to efficiently find occurrences of a word or phrase, enhancing search operations and user experience.

Plagiarism Detection: Employed by plagiarism detection software to compare documents and identify copied content by finding occurrences of similar text fragments.

Bioinformatics: Used for DNA and protein sequence analysis, enabling the matching of biological sequences to find specific gene sequences, mutations, or other relevant patterns.

Data Deduplication: Helps identify and eliminate duplicate data blocks in storage systems, optimizing storage use and performance.

Digital Forensics: Applied to search for specific patterns in large datasets, aiding investigations by identifying crucial evidence within digital data.

Network Intrusion Detection Systems: Leveraged to identify patterns in network traffic that might indicate malicious activity, helping to detect suspicious code snippets or command sequences in network traffic data packets.

Advantages And Disadvantages:

The Rabin-Karp algorithm is a string searching algorithm that uses hashing to find any one of a set of pattern strings in a text. Here are the key advantages and disadvantages of the Rabin-Karp algorithm:

Advantages

1. **Efficiency with Multiple Patterns**: Rabin-Karp is particularly efficient when searching for multiple patterns in the same text. By hashing the patterns and the substrings of the text, the algorithm can quickly identify potential matches.
2. **Simplicity**: The algorithm is relatively straightforward to implement and understand. It leverages simple hash functions and basic string operations, making it accessible.
3. **Performance on Average**: On average, the Rabin-Karp algorithm performs well with an expected time complexity of $O(n+m)$, where n is the length of the text and m is the length of the pattern. This makes it comparable to other string-matching algorithms.
4. **Use of Hashing**: Hashing allows for a quick comparison of the pattern with substrings of the text. Instead of comparing strings character by character, the algorithm compares hash values, which can be faster.

Disadvantages

1. **High Worst-Case Time Complexity**: In the worst case, the time complexity can degrade to $O(n \cdot m)$, especially if the hash function results in many collisions. This happens because every potential match requires a full string comparison to confirm.

2. **Hash Collisions**: The algorithm relies on hash functions, and collisions can occur (i.e., different strings producing the same hash value). Handling these collisions can complicate the implementation and impact performance.
3. **Hash Function Sensitivity**: The choice of a good hash function is crucial. A poor hash function can lead to many collisions, reducing the efficiency of the algorithm.
4. **Preprocessing Overhead**: For each substring of the text, a hash value needs to be computed, which adds some preprocessing overhead. While individual hash computations are generally fast, they can add up for very large texts.
5. **Memory Usage**: Maintaining hash values and dealing with potential collisions can require additional memory, which might be a concern in memory-constrained environments.

Summary

The Rabin-Karp algorithm is a powerful tool for string searching, particularly when dealing with multiple patterns. Its average-case efficiency and simplicity make it a useful choice in many scenarios. However, its worst-case performance and sensitivity to hash collisions and function choice mean it must be used with care, particularly in applications where performance guarantees are critical.

Algorithm:

1. **Initialization:**
    - **n** = length of the text.
    - **m** = length of the pattern.
    - **base** = 256 (or size of the character set).
    - **mod** = a prime number (used to avoid overflow and ensure good hash distribution).
    - **hpattern** = 0 (hash value for the pattern).
    - **htext** = 0 (hash value for the current window of text).
    - **h** = 1 (value of **base^(m-1) % mod**).
2. **Precompute the value of base^(m-1) % mod:**
    - For **i** from 0 to **m-2**:
        - **h = (h * base) % mod**.
3. **Compute the initial hash values for the pattern and the first window of the text:**
    - For **i** from 0 to **m-1**:
        - **hpattern = (base * hpattern + ord(pattern[i])) % mod**.
        - **htext = (base * htext + ord(text[i])) % mod**.
4. **Slide the pattern over the text one by one:**
    - For **i** from 0 to **n-m**:
        - If **hpattern == htext**:
            - Check the actual strings to confirm (to handle hash collisions).
            - If **text[i:i+m] == pattern**:

        - Record the index **i** (pattern found at this index).
    - If **i < n-m**:
        - Calculate the hash value for the next window of text:
            - **htext = (base * (htext - ord(text[i]) * h) + ord(text[i + m])) % mod**.
            - If **htext < 0**, adjust to make it positive:
                - **htext = htext + mod**.
5. **Return the list of indices where the pattern is found in the text.**

Pseudo Code:

```
Function RabinKarpSearch(Text, Pattern):
    n = Length(Text)
    m = Length(Pattern)
    h = Hash(Pattern)
    result = []

    for i = 0 to n - m:
        if Hash(Text[i:i+m]) == h:
            if Text[i:i+m] == Pattern:
                result.append(i)

    return result

Function Hash(S):
    // Compute a hash value for string S
    h = 0
    for each character in S:
        h = (h * base + ASCII(character)) % mod
    return h
```

Step by step process:

1. **Initialization:**
    - Compute the hash of the pattern and the initial hash of the text segment of the same length.
2. **Sliding Window:**
    - Slide the window one character at a time over the text.
    - Update the hash of the current text segment efficiently using a rolling hash technique.
3. **Comparison:**
    - If the hash of the current text segment matches the pattern hash, compare the actual strings to confirm a match.
4. **Update:**
    - Update the hash for the next segment by removing the leftmost character and adding the next character.
5. **Repeat:**
    - Repeat until the end of the text is reached.

Approach:

1. **Hashing:**
    - Use a hash function to convert the pattern and text segments into numerical values.
2. **Rolling Hash:**
    - Efficiently update the hash value as the window slides over the text.
3. **Comparison:**
    - Compare the hash values and confirm matches by direct string comparison to handle hash collisions.
4. **Optimization:**
    - Optimize the hash function and choose suitable parameters (base and modulus) to minimize collisions.

Rabin-Karp Algorithm Example:

Problem Statement:

Given a text "abracadabra" and a pattern "abra", use the Rabin-Karp algorithm to find all occurrences of the pattern in the text.

Rabin-Karp Algorithm Steps:

1. **Initialize Variables:**
    - Let n be the length of the text.
    - Let m be the length of the pattern.
    - Compute the hash values for the pattern and the first window of text.
2. **Hash Function:**
    - A simple hash function can be used, like polynomial rolling hash.
    - Compute hash value $H(s)$ for a string s as:

      $$H(s) = s[0] \cdot p^{m-1} + s[1] \cdot p^{m-2} + \ldots + s[m-1] \cdot p^0 \mod q$$

      where p is a prime number (chosen as 31 or 53), and q is a large prime modulus.
3. **Sliding the Pattern Over Text:**
    - Slide the pattern over text one by one and compare the hash values.
    - If the hash values match, perform character-by-character comparison to confirm the match.

Example Execution:

- **Text**: "abracadabra"
- **Pattern**: "abra"
- **Pattern Length (m)**: 4

- **Text Length (n)**: 11
- **Prime Number (p)**: 31
- **Modulus (q)**: 101

Detailed Steps:

1. **Compute Initial Hash Values:**
   - Compute hash of the pattern "abra":

$$H(\text{pattern}) = (97 \cdot 31^3 + 98 \cdot 31^2 + 114 \cdot 31^1 + 97 \cdot 31^0) \mod 101 = 4$$

   - Compute hash of the first window of text "abra":

$$H(\text{window}) = (97 \cdot 31^3 + 98 \cdot 31^2 + 114 \cdot 31^1 + 97 \cdot 31^0) \mod 101 = 4$$

2. **Compare and Slide:**
   - Compare the hash values:

   H (pattern) = 4 and H(window) = 4.

   - Since hash values match, compare characters:
     - "abra" matches "abra" at index 0.
3. **Slide the Window and Recompute Hash:**
   - Slide window to the right and compute hash for "brac":

$$H(\text{new window}) = ((H(\text{previous window}) - 97 \cdot 31^3) \cdot 31 + 99) \mod 101 = 30$$

   - Compare H(pattern) = 4 with H(new window) = 30. No match.
4. **Continue Sliding and Checking:**
   - Repeat the process for subsequent windows:
     - "raca" -> Hash = 55, no match.
     - "acad" -> Hash = 96, no match.
     - "cada" -> Hash = 3, no match.

- "adab" -> Hash = 34, no match.
- "dabr" -> Hash = 7, no match.
- "abra" -> Hash = 4, match found at index 7.

5. **Pattern Occurrences:**
    - The pattern "abra" is found at positions 0 and 7.

CODE

code.py

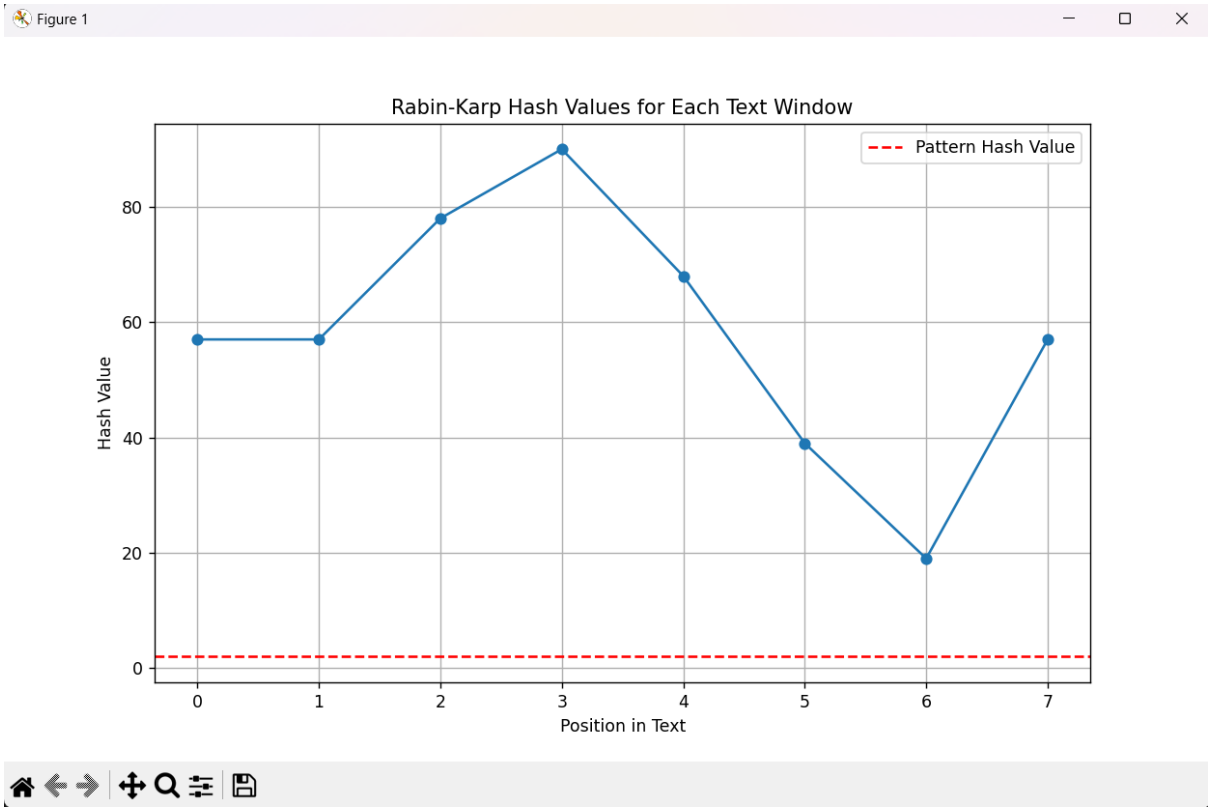
C: > Users > Admin > Desktop > Bhanvanu > Sem4 > DAA > Lab Report > code.py > ...

```python
import matplotlib.pyplot as plt

def rabin_karp_search_with_visualization(text, pattern):
    n = len(text)
    m = len(pattern)
    p = 31  # Prime number for hash computation
    q = 101  # Large prime modulus
    p_m = pow(p, m-1, q)  # p^(m-1) % q

    pattern_hash = 0
    text_hash = 0

    for i in range(m):
        pattern_hash = (pattern_hash * p + ord(pattern[i])) % q
        text_hash = (text_hash * p + ord(text[i])) % q

    matches = []
    hash_values = []

    for i in range(n - m + 1):
        hash_values.append((i, text_hash))
        if pattern_hash == text_hash:
            if text[i:i + m] == pattern:
                matches.append(i)

        if i < n - m:
            text_hash = (text_hash - ord(text[i]) * p_m) * p + ord(text[i + m])
            text_hash = text_hash % q

    return matches, hash_values

text = "abracadabra"
pattern = "abra"
matches, hash_values = rabin_karp_search_with_visualization(text, pattern)

# Print the result
print("Pattern found at positions:", matches)

# Plotting the hash values
positions, values = zip(*hash_values)

plt.figure(figsize=(10, 6))
plt.plot(positions, values, marker='o')
plt.axhline(y=sum(ord(c) for c in pattern) % 101, color='r', linestyle='--', label='Pattern Hash Value')
plt.title('Rabin-Karp Hash Values for Each Text Window')
plt.xlabel('Position in Text')
plt.ylabel('Hash Value')
plt.legend()
plt.xticks(positions)
plt.grid(True)
plt.show()
```

Output:

```
PS C:\Users\Admin> & C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Admin/Desktop/Bhanvanu/Sem4/DAA/Lab Report/code.py"
Pattern found at positions: [0, 7]
PS C:\Users\Admin>
```

Graph:



Analysis:

The Rabin-Karp Algorithm is a method used to search for a pattern in a text, leveraging hashing to efficiently compare segments of the text with the pattern. This approach is particularly useful for applications requiring fast pattern matching. Let's take a closer look at its approach:

1. **Starting Point:** Imagine you have a large body of text, such as a book, and you want to find all occurrences of a specific word or phrase. The text is analogous to a field of hay, and the pattern you're searching for is the needle. The Rabin-Karp Algorithm helps you efficiently sift through the hay to find the needle.
2. **Hashing the Pattern:** The first step in the Rabin-Karp Algorithm is to compute a hash value for the pattern you're searching for. Hashing converts the pattern into a numerical value, which simplifies the process of comparison. Think of this hash value as a unique identifier for the pattern.
3. **Sliding Window:** Next, the algorithm applies a sliding window technique to the text. This means it starts at the beginning of the text and looks at a segment the same length as the pattern. It then computes the hash value for this segment. If the hash values match, there's a potential match for the pattern.
4. **Efficient Comparison:** To make this process efficient, the algorithm uses a rolling hash technique. As the window slides one character to the right, the hash value is updated by removing the contribution of the outgoing character and adding the contribution of the incoming character. This update can be done in constant time, O(1), making the comparison process very fast.
5. **Direct Verification:** When the hash values of the current text segment and the pattern match, the algorithm performs a direct string comparison to verify that the segment indeed matches the

pattern. This step ensures that any hash collisions (different strings producing the same hash value) are handled correctly.

6. **Building Up:** The algorithm continues sliding the window and updating the hash value until it has checked every possible position in the text where the pattern could fit. Each time a match is found, the position is recorded.
7. **Repeating the Process:** For multiple patterns, the Rabin-Karp Algorithm can be repeated or adjusted to handle multiple hashes simultaneously, making it versatile for various applications.
8. **Final Result:** Once the entire text has been processed, the algorithm provides a list of all positions where the pattern was found. This list allows for quick identification of all occurrences of the pattern within the text.

Conclusion:

The Rabin-Karp Algorithm is a powerful tool for pattern matching in strings, leveraging the efficiency of hashing and rolling hash techniques. Its ability to quickly identify potential matches and confirm them with direct comparison makes it ideal for a wide range of applications, from text searching to bioinformatics. By understanding and implementing this algorithm, developers and researchers can efficiently tackle problems involving large datasets and complex pattern matching tasks.

The time complexity of the Rabin-Karp Algorithm is O(n) in the average case, where n is the length of the text, assuming a good hash function. In the worst-case scenario, the complexity can degrade to O(mn), where m is the length of the pattern, but this is rare with a well-chosen hash function and parameters.

References:

https://www.researchgate.net/publication/332773245_Exact_String_Matching_Algorithms_Survey_Issues_and_Future_Research_Directions

https://www.semanticscholar.org/paper/Implementation-of-Rabin-Karp-String-Matching-Using-Kohli/d70117bbc40fb185ae3ac392d67028cc506727d1#extracted